ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмної інженерії»

ЗВІТ

з лабораторної роботи № 1

з дисципліни «Технології розробки гібридних систем штучного інтелекту»

Виконав (ла)
ст. гр. ІПЗм-26-2
Сидоренко Катерина

Прийняв (ла): Дашенков Д. С.

Харків 2026

1 ЛАБОРАТОРНА РОБОТА № 1. РОЗРОБЛЕННЯ RAG-СИСТЕМИ. ПРОЄКТУВАННЯ ПЗ

1.1 Мета роботи та постановка завдання

Метою роботи є набуття практичних навичок попереднього проєктування програмного забезпечення для RAG-системи. Зокрема, потрібно навчитися формулювати компактний сценарій використання, свідомо обирати дані та технології, зображати архітектуру системи і готувати робочий простір проєкту.

У цьому звіті подано перші два завдання. Перше полягає у формулюванні сценарію використання обсягом близько однієї сторінки, який охоплює цільового користувача, типове запитання та критерії гарної й поганої відповіді. Друге передбачає вибір корпусу від п'ятисот до десяти тисяч документів з документуванням назви та джерела набору, ліцензії й кількості документів. Вибір стеку, архітектурну діаграму та репозиторій буде описано після їх виконання.

1.2 Хід виконання роботи

1.2.1 Завдання 1. Сценарій використання

1.2.1.1 Опис сценарію

Система призначена для пошуку книжки за її описом. Читач пам'ятає сюжет, героїв або окремі епізоди, але не може згадати ні назви, ні автора. Він описує книжку своїми словами, а система знаходить у корпусі найближчі описи й повертає назву, автора та уривок, який підтверджує відповідь.

Цільовим користувачем є звичайний читач без технічної підготовки, який працює з системою через командний рядок. Він може пам'ятати книжку з дитинства або згадати її через розмову з друзями. Типовий запит виглядає так: «A detective novel where a murder happens on a train and a Belgian detective with a big moustache investigates». Користувач пам'ятає не все, тому в описі можуть бути неточності, пропущені імена чи переплутані деталі. Система має це враховувати.

1.2.1.2 Обмеження сценарію

Щоб сценарій був придатний для перевірки, його межі зафіксовано в таблиці 1.1.

Таблиця 1.1 Обмеження сценарію

Параметр	Обмеження
Мова запитів і відповідей	Лише англійська. Запити іншими мовами не підтримуються, і результат для них не гарантується.
Мова корпусу	Англійська.
Вхід	Один текстовий опис книжки. Діалог з уточненнями не передбачено.
Інтерфейс	Командний рядок. Один запит дає одну відповідь.
Джерело знань	Лише відібрана підмножина корпусу. Знання мовної моделі про книжки поза корпусом не використовуються.
Вихід	До трьох книжок. Для кожної вказуються назва, автор і рік публікації, якщо вони є в записі, а також уривок опису, на якому ґрунтується відповідь.
Відсутня книжка	Якщо відповідного опису в корпусі немає, система повідомляє, що книжку не знайдено, і не пропонує припущень.
Поза межами системи	Рекомендації та оцінки книжок, пошук за обкладинкою чи цитатою з тексту, посилання на купівлю, повні тексти книжок.

1.2.1.3 Критерії якості відповіді

Відповідь вважається гарною, якщо виконано всі наведені нижче умови:

- правильна книжка є серед запропонованих кандидатів, а найкраще, якщо стоїть першою;

- кожне твердження підтверджене уривком із корпусу, на який є посилання у відповіді;

- назва, автор і рік збігаються із записом у корпусі;

- якщо впевненості немає, система показує до трьох кандидатів з коротким поясненням або повідомляє, що книжки не знайдено.

Відповідь вважається поганою, якщо трапляється хоча б одна з таких ситуацій:

- названо книжку чи автора, яких немає в корпусі, або вигадано сюжет;

- відповідь не містить посилання на знайдений уривок;

- на запит про книжку, якої немає в корпусі, система впевнено називає іншу;

- назва, автор чи рік не збігаються із записом;

- кандидатів більше трьох, і користувач не може відрізнити найімовірнішого.

Для кожного запитання існує одна еталонна книжка, тому якість можна оцінювати автоматично.

1.2.2 Завдання 2. Вибір корпусу даних

1.2.2.1 Вимоги до корпусу

Спершу сформульовано вимоги до корпусу, виходячи зі сценарію та умов завдання. Вони такі:

1) обсяг від 500 до 10 000 документів;

2) документом є опис сюжету, за яким книжку можна впізнати, разом із назвою та автором, бо без цього сценарій неможливо ні реалізувати, ні перевірити;

3) англійська мова, відповідно до обмежень сценарію;

4) відкрита ліцензія, що дозволяє повторне розповсюдження, оскільки код і супутні матеріали публікуються у відкритому репозиторії;

5) офіційне першоджерело, за яким можна відновити походження даних і прочитати умови використання;

6) наявність жанру бажана, але не обов'язкова.

1.2.2.2 Порівняння кандидатів

Для порівняння розглянуто шість кандидатів. Дані про ліцензії та поля взято зі сторінок самих наборів. Результат наведено в таблиці 1.2.

Таблиця 1.2 Порівняння кандидатів на роль корпусу

Кандидат	Ліцензія та параметри	Висновок
Goodreads-Books від Bright Data [1]	Ліцензія «other», власна угода Bright Data. Понад 6,3 млн записів, 14 полів.	Не підходить. Ліцензія не є відкритою, а обсяг значно перевищує потрібний.
gutendex-dataset на Hugging Face [2]	Ліцензію в картці не вказано. 26 138 записів. Набір створено на основі каталогу Gutendex, що описує книжки Project	Не підходить. Без явної ліцензії діють звичайні авторські права, а наявність описів сюжетів у картці не

	Gutenberg.	підтверджена.
Book Summary Dataset Notebook на Kaggle [3]	Це ноутбук, а не окремий набір даних. Ліцензію першоджерела не підтверджено. У даних відсутній автор книжки.	Не підходить. Немає автора, а походження даних не підтверджене.
cmu-book-summaries, дзеркало на Hugging Face [4]	Ліцензія «cc» без версії. 16 559 записів. Поля: назва, автор, рік, опис. Жанрів немає, походження в картці не описане.	Придатне лише для перегляду. Ліцензія й походження неточні, тому використовуємо першоджерело.
CMU Book Summary Dataset, сторінка авторів [5]	CC BY-SA 3.0 US. 16 559 книжок. Назва, автор, дата, жанри, опис сюжету. Описи взято з Вікіпедії, метадані з Freebase.	Задовольняє всі вимоги. Обрано.
Англійська Вікіпедія, набір wikimedia/wikipedia, конфігурація 20231101.en [6]	cc-by-sa-3.0 і gfdl. Поля: id, url, title, text. Умови повторного використання описано на сторінці Вікіпедії [7].	Ліцензія підходить, але це статті на довільні теми без категорій і жанрів. Статті про книжки довелося б відбирати за назвою чи вмістом, а сюжет виділяти з тексту. Тому це запасний варіант.

Дзеркало набору CMU на Hugging Face [4] зручне для швидкого перегляду, але його картка не описує походження даних і не вказує версію ліцензії. Тому за першоджерело взято сторінку авторів набору.

1.2.2.3 Обраний корпус

Обрано CMU Book Summary Dataset. Він єдиний із розглянутих одночасно містить стислі описи сюжетів, назву й автора, має чітко заявлену відкриту ліцензію та перебуває на сторінці авторів, де зазначено походження даних. Англійська Вікіпедія є першоджерелом цих описів, але її статті не розмічені за жанрами, і вибір книжкових сторінок разом із виділенням сюжету довелося б робити самостійно. У наборі CMU цю роботу вже виконано. Додатковою перевагою є жанри, які можна використати для фільтрації.

Паспорт обраного корпусу наведено в таблиці 1.3.

Таблиця 1.3 Паспорт корпусу

Параметр	Значення
Назва	CMU Book Summary Dataset
Джерело	Сторінка авторів набору в Університеті Карнегі-Меллона [5]

Автори і публікація	D. Bamman, N. A. Smith. New Alignment Methods for Discriminative Book Summarization, 2013 [8]
Походження даних	Описи сюжетів вилучено з Вікіпедії, метадані зіставлено з Freebase.
Ліцензія	Creative Commons Attribution-ShareAlike (CC BY-SA 3.0 US), як заявлено авторами набору.
Кількість документів	16 559 книжок у повному наборі. Для проєкту планується підмножина, орієнтовно 5 000 книжок.
Поля запису	Ідентифікатори Вікіпедії та Freebase, назва, автор, дата публікації, жанри, опис сюжету.
Мова	Англійська.

Дані в таблиці 1.3 взято зі сторінки набору.

1.2.2.4 Ліцензія корпусу

Описи сюжетів у наборі є текстом Вікіпедії, а авторство в Вікіпедії колективне. Ліцензія CC BY-SA не забороняє ні навчання, ні експерименти, ні комерційне використання. Вона вимагає, щоб той, хто поширює такі дані чи їх змінену версію, вказав авторство, позначив зміни й поширював результат на тих самих умовах [7]. Якщо виникне потреба опублікувати похідні матеріали, зокрема відібрану підмножину чи розбиті на фрагменти тексти, до них додаватимуться повідомлення про ліцензію CC BY-SA, посилання на джерело та опис внесених змін.

1.2.3 Завдання 3. Вибір технологічного стеку

Система складається з кількох компонентів, які працюють у два етапи. На етапі підготовки пайплайн прийому даних завантажує й очищує корпус, модель ембедингів перетворює описи книжок на вектори, а векторний індекс зберігає їх разом з метаданими. На етапі відповіді користувач вводить запит в інтерфейсі, ретривер кодує його тією самою моделлю ембедингів і знаходить в індексі найближчі описи, а модель генерації формує з них відповідь із посиланням на уривок.

Якість кожної ланки впливає на наступну. Ретривер не знайде того, що погано проіндексовано, а модель генерації не додасть знань, яких немає в знайдених описах.

1.2.3.1 Векторне сховище

Векторне сховище зберігає вектори описів книжок разом із супутніми даними та за запитом повертає найближчі до вектора запиту. Від нього залежить, чи швидко запускатиметься система з командного рядка, чи зможе вона вивести назву й автора книжки та чи вдасться відрізнити впевнену відповідь від невпевненої. З огляду на сценарій до сховища висунуто п'ять вимог.

Локальність означає, що сховище працює на тому самому комп'ютері, що й система, без окремого сервера чи хмарного сервісу.

Збереження метаданих означає здатність зберігати поруч із вектором назву, автора, рік, жанри та текст опису. Від цього залежить, чи зможе система вивести структуровану відповідь і перевірити, що цитований ідентифікатор належить до знайдених записів.

Персистентність означає, що індекс зберігається на диску та завантажується без перебудови. Індекс будується один раз з усього корпусу, а потім використовується під час кожного запуску.

Повернення схожості означає, що сховище повертає не лише список знайдених записів, а й числову відстань або оцінку близькості для кожного. Це визначає поведінку системи, коли потрібної книжки в корпусі немає. За значенням схожості можна задати поріг, нижче якого система відповідає, що книжку не знайдено, замість того щоб називати найближчу, але хибну. Також від цього залежить порядок кандидатів у відповіді.

Малий масштаб означає, що корпус налічує орієнтовно 5 000 описів, тому розподілений пошук, шардинг і складні наближені індекси не потрібні. Простота розгортання важливіша за граничну продуктивність.

Для вибору розглянуто п'ять відкритих рішень, а саме Chroma [9], FAISS [10], Qdrant [11], pgvector [12] та Vespa [13]. Їх порівняння за наведеними вимогами подано в таблиці 1.4.

Таблиця 1.4 – Порівняння векторних сховищ

Вимога	Chroma	FAISS	Qdrant	pgvector	Vespa

Локальність	Так. Пакет chromadb для Python[9]	Так. Бібліотека, яка працює в процесі програми [10]	Частково. Сервер у контейнері Docker [11]	Частково. Потрібен локальний сервер PostgreSQL [12]	Частково. Контейнер Docker, не менше 4 ГБ ОЗП, окреме розгортання застосунку [13]
Збереження метаданих	Так. Зберігає документи й метадані, фільтрує за ними [14]	Ні. Лише вектори й цілі ідентифікатори [10]	Так. Довільний JSON у payload разом із вектором [15]	Так. Вектор є стовпцем таблиці поруч з іншими полями [12]	Так. Документи зі структурованими полями, потрібна схема застосунку [13]
Персистентність	Так. PersistentClient зберігає дані в каталозі й завантажує під час старту [16]	Так, вручну. Окремі виклики збереження й завантаження індексу [17]	Так. Volume [11]	Так. Дані в PostgreSQL із журналом WAL [12]	Так. Дані зберігаються в контейнері, потрібне місце на диску [13]
Повернення схожості	Так. Запит повертає відстані разом із документами [14]	Так. Повертає матрицю відстаней та ідентифікатори [17]	Так. Пошук за заданою метрикою відстані [11]	Так. Оператори косинусної, евклідової відстані та скалярного добутку [12]	Так. Оцінка релевантності, наприклад BM25 [13]
Малий масштаб	Підходить. Мінімум налаштувань	Підходить, але потрібен додатковий код	Надмірно для кількох тисяч записів	Надмірно. Адміністрування бази даних заради одного індексу	Надмірно. Найбільш громіздке розгортання
Підсумок	Обрано	Не обрано. Немає метаданих	Не обрано. Потрібен окремий сервер	Не обрано. Потрібен PostgreSQL	Не обрано. Надто громіздка

Обрано Chroma. Вона єдина з розглянутих задовольняє всі п'ять вимог без додаткових сервісів і допоміжного коду. Вона працює локально, зберігає метадані поруч із векторами, сама записує індекс на диск і повертає відстані до знайдених описів. FAISS швидка, але змусила б вести окрему таблицю метаданих. Qdrant, pgvector і Vespa вимагають окремого сервера, що для корпусу на кілька тисяч записів зайве. Обрана Chroma розрахована на невеликі обсяги, тому якщо корпус суттєво зросте, вибір доведеться переглянути.

1.2.3.2 Модель ембедингів

Модель ембедингів перетворює текст на вектор, у якому близькі за змістом тексти лежать поруч. Саме вона визначає, що система вважатиме схожим, тому від неї залежить, чи знайде ретривер правильну книжку за нечітким описом. Для запитів і для корпусу потрібна одна й та сама модель, а після її заміни індекс доведеться будувати заново. З огляду на сценарій до моделі висунуто п'ять вимог.

Англомовність означає, що модель навчена на англійських текстах. Корпус і запити за сценарієм англійські, тому вимога прямо випливає з обмежень проєкту. Якщо взяти модель для іншої мови, вектори для англійських слів можуть бути неякісними, і пошук втратить сенс.

Безкоштовність і локальний запуск означають, що модель має відкриту ліцензію й працює на звичайному ноутбуці без відеокарти та без платного API. Розмір моделі впливає на швидкість запуску, адже командний рядок завантажує її на кожен запит. Корпус кодується один раз, але запит кодується щоразу.

Довжина вхідного тексту означає максимум токенів, які модель бере до уваги. Все, що довше, обрізається і не впливає на вектор. Описи сюжетів можуть бути довгими, тому деталі з кінця опису за малого ліміту система не побачить, а користувач може згадати саме їх.

Розмірність вектора й формат входу означають кількість чисел у векторі та вимоги до підготовки тексту. Розмірність впливає на розмір індексу та сумісність зі сховищем. Вимоги до формату входу, наприклад обов'язкові префікси, впливають на складність коду і ризик помилки, адже один пропущений префікс може погіршити якість пошуку.

Для вибору розглянуто три компактні англомовні моделі, а саме all-MiniLM-L6-v2 [18], bge-small-en-v1.5 [19] та e5-small-v2 [20]. Їх порівняння за наведеними вимогами подано в таблиці 1.5.

Таблиця 1.5 Порівняння моделей ембедингів

Вимога	all-MiniLM-L6-v2	bge-small-en-v1.5	e5-small-v2
Англомовність	Так, мова англійська [18]	Так, мова англійська [19]	Так, працює лише з англійськими текстами [20]

Ліцензія й розмір	Apache-2.0, 22,7 млн параметрів [18]	MIT, 33,4 млн параметрів [19]	MIT, 33,4 млн параметрів [20]
Довжина вхідного тексту	Текст довший за 256 токенів обрізається [18]	До 512 токенів [19]	До 512 токенів, довший текст обрізається [20]
Розмірність і формат входу	384, додаткових префіксів немає [18]	384, префікс для запиту необов'язковий [19]	384, префікси «query: » і «passage: » обов'язкові [20]
Підсумок	Запасний варіант	Обрано	Не обрано. Складніший формат входу

Обрано bge-small-en-v1.5. Вона обробляє тексти до 512 токенів, тобто вдвічі більше за all-MiniLM-L6-v2, має ліцензію MIT і підтримується бібліотекою sentence-transformers. Префікс для запиту в неї необов'язковий, тому код простіший, ніж з e5-small-v2, де префікси потрібні завжди.

1.2.3.3 Модель генерації

Модель генерації отримує запит і знайдені описи книжок та формує з них відповідь, тобто назву, автора й пояснення з посиланням на уривок. Нових знань вона не додає, але від неї залежить, чи буде відповідь у потрібному форматі, чи цитуватиме вона джерела і чи відмовиться відповідати, коли потрібного опису в контексті немає. Для вибору враховано п'ять вимог.

Локальний запуск і безкоштовність означають, що модель працює на ноутбуці без платного API.

Довжина контексту означає кількість токенів, які модель обробляє за один запит. У неї мають поміститися інструкція, запит користувача та до трьох знайдених описів. Якщо контекст замалий, описи доведеться обрізати, і система втратить частину інформації, на якій ґрунтується відповідь.

Дотримання інструкцій і формату виходу означає здатність відповідати у заданій структурі, цитувати джерела за номерами та відмовлятися від відповіді за браку контексту. Від цього залежить, чи зможе код розібрати відповідь і перевірити, що цитовані ідентифікатори належать знайденим записам, а також чи не вигадуватиме модель книжок, яких немає в корпусі.

Англомовність і ліцензія означають, що модель підтримує англійську мову й має умови використання, які дозволяють навчальний проєкт з

відкритим кодом. Вимога впливає на якість розуміння англомовних описів та на те, чи можна публікувати результат у репозиторії.

Для вибору розглянуто шість варіантів, а саме моделі Llama 3.2 3B [21], Qwen 2.5 7B [22], які запускаються через Ollama [23], безкоштовний рівень Google AI Studio з моделями Gemini Flash [24] та безкоштовні моделі OpenRouter [25]. Їх порівняння подано в таблиці 1.6.

Таблиця 1.6 Порівняння варіантів моделі генерації

Варіант	Запуск і вартість	Розмір і контекст	Ліцензія й умови	Висновок
Llama 3.2 3B через Ollama	Локально, безкоштовно. Ollama має ліцензію MIT [23]	Завантаження 2,0 ГБ, контекст 128 тис. токенів [21]	Llama 3.2 Community License [26]. Англійська входить до восьми офіційних мов [21]	Обрано
Qwen 2.5 7B через Ollama	Локально, безкоштовно [23]	Завантаження 4,7 ГБ, у переліку контекст 32 тис. токенів [22]	Apache 2.0 для 7B. Покращено генерацію структурованого виходу, зокрема JSON [22]	Запасний варіант для потужнішого ноутбука
Google AI Studio, Gemini Flash	Хмара, безкоштовний рівень. Потрібен ключ доступу [24]	Ліміти запитів безкоштовного рівня на сторінці не наведено [24]	Вміст безкоштовного рівня використовується для покращення продуктів Google [24]	Не обрано. Залежність від хмари та умови використання даних
Безкоштовні моделі OpenRouter	Хмара. Деякі моделі мають безкоштовні варіанти [25]	До 50 запитів на добу без придбання кредитів [25]	Промпти не логуються за замовчуванням [25]	Не обрано. Ліміт замалий для тестування

Обрано Ollama з моделлю Llama 3.2 3B. Це найменша з розглянутих локальних моделей, її завантаження займає близько 2 ГБ, тому вона працюватиме на звичайному ноутбуці та швидко відповідатиме на процесорі. Контекст у 128 тисяч токенів з великим запасом вміщує інструкцію й три описи, а англійська входить до офіційно підтримуваних мов. Ollama має ліцензію MIT, REST API та клієнт для Python. Хмарні варіанти відхилено через ліміти запитів і умови використання даних. Достатність якості Llama 3.2 3B підтвердить тестування. Ліцензія Llama 3.2 вимагає позначки «Built with Llama» у разі поширення матеріалів на її основі, тому це буде враховано під час публікації проєкту.

1.2.3.4 Мова програмування

Мова програмування визначає, якими бібліотеками можна скористатися і наскільки складним буде розгортання. Весь код проєкту, тобто пайплайн даних, ретривер, взаємодія з моделлю та командний рядок, пишеться однією мовою, тому вибір впливає на всю роботу. До мови висунуто чотири вимоги.

Підтримка обраних бібліотек означає, що Chroma, бібліотека для моделі ембедингів і клієнт Ollama доступні мовою без обхідних рішень. Інакше довелося б писати власні обгортки, що збільшить обсяг коду й кількість помилок.

Робота без окремого сервера означає, що сховище та модель ембедингів працюють в одному процесі зі скриптом. Це впливає на швидкість запуску та простоту розгортання, адже командний рядок стартує заново на кожен запит.

Засоби для командного рядка означають наявність стандартних засобів для читання аргументів запиту та виведення структурованої відповіді. Від цього залежить кількість додаткових залежностей.

Відтворюваність і ліцензія означають, що мова вільна для використання, а залежності встановлюються однією командою зі зафіксованими версіями. Вимога впливає на те, чи зможе викладач відтворити результат на своїй машині.

Для вибору розглянуто Python і JavaScript із TypeScript. Версії Python, які потрібні обраним бібліотекам, різняться. Chroma [27] потребує Python 3.9 або новішого, sentence-transformers [28] потребує 3.10, а клієнт Ollama [29] працює з 3.8. Тому для всього стеку потрібен Python 3.10 або новіший. Порівняння подано в таблиці 1.7.

Таблиця 1.7 Порівняння мов програмування

Вимога	Python	JavaScript і TypeScript
Підтримка обраних бібліотек	Chroma, sentence-transformers і клієнт Ollama розповсюджуються як пакети Python [27] [28] [29]	Клієнт Ollama існує [23]. Chroma має клієнт для TypeScript, але для збереження даних потрібен сервер [16]
Робота без окремого сервера	PersistentClient зберігає дані на диск у процесі скрипту [16]	Збереження досягається запуском локального сервера [16]
Засоби командного рядка	Модуль argparse входить до стандартної бібліотеки [30]	Не оцінювалося, оскільки попередні вимоги не виконано

Відтворюваність і ліцензія	Ліцензія PSF, Python є відкритим [31].	Не оцінювалося, оскільки попередні вимоги не виконано
Висновок	Обрано	Не обрано

Обрано Python версії 3.10 або новішої. Лише в ньому всі три компоненти, а саме Chroma, sentence-transformers та клієнт Ollama, працюють в одному процесі без окремого сервера. Для командного рядка вистачає модуля argparse зі стандартної бібліотеки. Залежності встановлюватимуться через pip, а їхні версії буде зафіксовано у файлі залежностей репозиторію.

1.2.4 Завдання 4. Архітектурна діаграма системи

1.2.4.1 Вибір виду діаграми

Для опису архітектури обрано блок-схему компонентів, яка показує склад системи та напрямки передавання даних між ними. Такий вид діаграми підходить, бо завдання полягає у визначенні компонентів і зв'язків між ними, а не порядку викликів у часі чи структури коду.

1.2.4.2 Склад і призначення компонентів

Архітектуру системи подано на рисунку 1.1. Вона складається з шести компонентів і двох зовнішніх сутностей, а саме корпусу даних і користувача. Компоненти розділено на дві групи за фазами роботи.

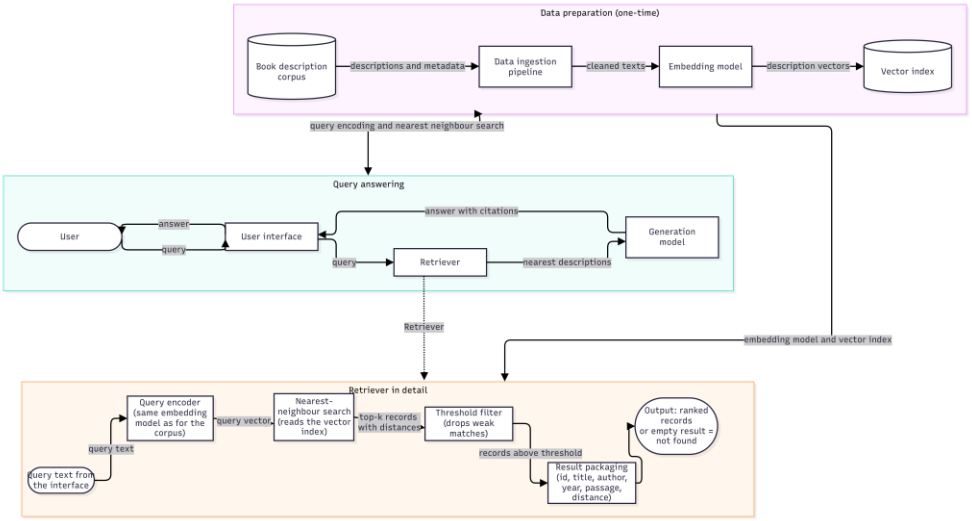


Рисунок 1.1 – Архітектурна діаграма системи

Призначення компонентів:

- пайплайн прийому даних завантажує корпус, відбирає записи, очищує тексти та формує для кожної книжки запис з ідентифікатором, назвою, автором, роком, жанрами й текстом опису;

- модель ембедингів перетворює тексти на вектори, причому як описи книжок, так і запити користувача, щоб їх можна було порівнювати між собою;

- векторний індекс зберігає вектори описів разом із метаданими та за запитом повертає найближчі записи з відстанями до них;

- інтерфейс користувача приймає запит із командного рядка, перевіряє його довжину та показує відповідь;

- ретривер кодує запит, шукає в індексі найближчі описи й передає їх далі разом зі значеннями схожості, за якими можна визначити, що відповідного опису немає;

- модель генерації отримує запит і знайдені описи та формує відповідь із цитатами уривків або повідомляє, що книжку не знайдено.

1.2.5 Завдання 5. Ініціалізація репозиторію

1.2.5.1 Структура репозиторію

Для проєкту створено публічний репозиторій rag-project-sydorenko на платформі GitHub.

1.2.5.2 Файл README

У корені репозиторію розміщено файл README.md, який дає змогу зрозуміти проєкт без звернення до звіту. Він містить такі розділи:

- назву проєкту й короткий опис;

- сценарій використання з обмеженнями системи;

- опис корпусу з посиланням на джерело та ліцензією;

- таблицю обраних технологій;

- покрокову інструкцію встановлення;

- структуру репозиторію;

- таблицю етапів проєкту з відповідними тегами.

Інструкція встановлення охоплює перевірку версії Python, клонування репозиторію, створення віртуального середовища, встановлення бібліотек Chroma, sentence-transformers і клієнта Ollama, встановлення самого Ollama та завантаження моделі генерації. Версії бібліотек у ній поки не зафіксовано. Це буде зроблено на етапі підготовки даних.

1.2.5.3 Тег і посилання на репозиторій

Посилання на репозиторій:
https://github.com/NureSydorenkoKateryna/rag-project-sydorenko

1.3 Висновки

У ході лабораторної роботи набуто практичних навичок попереднього проєктування програмного забезпечення для RAG-системи. Сформульовано сценарій використання, межі якого визначено обмеженнями щодо мови, довжини запиту та формату відповіді. Обрано корпус англомовних описів книжок і розглянуто умови його ліцензії. Для векторного сховища, моделі ембедингів, моделі генерації та мови програмування зроблено вибір з обґрунтуванням. Архітектуру системи зображено на діаграмі, а робочий простір проєкту підготовлено в репозиторії та зафіксовано тегом lab-1.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. BrightData/Goodreads-Books [Електронний ресурс] // Hugging Face Datasets. URL: https://huggingface.co/datasets/BrightData/Goodreads-Books (дата звернення: 29.10.2026).

2. AvishayDev/gutendex-dataset [Електронний ресурс] // Hugging Face Datasets. URL: https://huggingface.co/datasets/AvishayDev/gutendex-dataset (дата звернення: 29.10.2026).

3. Book Summary Dataset Notebook [Електронний ресурс] / arpansri (автор облікового запису) // Kaggle. URL: https://www.kaggle.com/code/arpansri/book-summary-dataset-notebook/notebook (дата звернення: 29.10.2026).

4. textminr/cmu-book-summaries [Електронний ресурс] // Hugging Face Datasets. URL: https://huggingface.co/datasets/textminr/cmu-book-summaries (дата звернення: 29.10.2026).

5. CMU Book Summary Dataset [Електронний ресурс] / D. Bamman. Carnegie Mellon University. URL: https://www.cs.cmu.edu/~dbamman/booksummaries.html (дата звернення: 29.10.2026).

6. wikimedia/wikipedia [Електронний ресурс] // Hugging Face Datasets. URL: https://huggingface.co/datasets/wikimedia/wikipedia (дата звернення: 29.10.2026).

7. Wikipedia:Reusing Wikipedia content [Електронний ресурс] // Wikipedia. URL: https://en.wikipedia.org/wiki/Wikipedia:Reusing_Wikipedia_content (дата звернення: 29.10.2026).

8. Bamman D., Smith N. A. New Alignment Methods for Discriminative Book Summarization [Електронний ресурс] // arXiv:1305.1319. 2013. URL: https://arxiv.org/abs/1305.1319 (дата звернення: 29.10.2026).

9. Getting Started [Електронний ресурс] // Chroma Docs. URL: https://docs.trychroma.com/docs/overview/getting-started (дата звернення: 29.10.2026).

10. Faiss: a library for efficient similarity search and clustering of dense vectors [Електронний ресурс] // GitHub, Meta Research. URL: https://github.com/facebookresearch/faiss (дата звернення: 29.10.2026).
11. Local Quickstart [Електронний ресурс] // Qdrant Documentation. URL: https://qdrant.tech/documentation/quickstart/ (дата звернення: 29.10.2026).
12. pgvector: Open-source vector similarity search for Postgres [Електронний ресурс] // GitHub. URL: https://github.com/pgvector/pgvector (дата звернення: 29.10.2026).
13. Text Search Tutorial [Електронний ресурс] // Vespa Documentation. URL: https://docs.vespa.ai/en/learn/tutorials/text-search.html (дата звернення: 29.10.2026).
14. Query and Get [Електронний ресурс] // Chroma Docs. URL: https://docs.trychroma.com/docs/querying-collections/query-and-get (дата звернення: 29.10.2026).
15. Payload [Електронний ресурс] // Qdrant Documentation. URL: https://qdrant.tech/documentation/manage-data/payload/ (дата звернення: 29.10.2026).
16. Clients [Електронний ресурс] // Chroma Docs. URL: https://docs.trychroma.com/docs/run-chroma/clients (дата звернення: 29.10.2026).
17. Getting started [Електронний ресурс] // Faiss Wiki, GitHub. URL: https://github.com/facebookresearch/faiss/wiki/Getting-started (дата звернення: 29.10.2026).
18. sentence-transformers/all-MiniLM-L6-v2 [Електронний ресурс] // Hugging Face. URL: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (дата звернення: 29.10.2026).
19. BAAI/bge-small-en-v1.5 [Електронний ресурс] // Hugging Face. URL: https://huggingface.co/BAAI/bge-small-en-v1.5 (дата звернення: 29.10.2026).
20. intfloat/e5-small-v2 [Електронний ресурс] // Hugging Face. URL: https://huggingface.co/intfloat/e5-small-v2 (дата звернення: 29.10.2026).

21. llama3.2 [Електронний ресурс] // Ollama Library. URL: https://ollama.com/library/llama3.2 (дата звернення: 29.10.2026).
22. qwen2.5 [Електронний ресурс] // Ollama Library. URL: https://ollama.com/library/qwen2.5 (дата звернення: 29.10.2026).
23. Ollama: Get up and running with large language models [Електронний ресурс] // GitHub. URL: https://github.com/ollama/ollama (дата звернення: 29.10.2026).
24. Gemini Developer API pricing [Електронний ресурс] // Google AI for Developers. URL: https://ai.google.dev/gemini-api/docs/pricing (дата звернення: 29.10.2026).
25. FAQ [Електронний ресурс] // OpenRouter Documentation. URL: https://openrouter.ai/docs/faq (дата звернення: 29.10.2026).
26. Llama 3.2 Community License Agreement [Електронний ресурс] // Meta. URL: https://www.llama.com/llama3_2/license/ (дата звернення: 29.10.2026).
27. chromadb [Електронний ресурс] // Python Package Index. URL: https://pypi.org/project/chromadb/ (дата звернення: 29.10.2026).
28. sentence-transformers [Електронний ресурс] // Python Package Index. URL: https://pypi.org/project/sentence-transformers/ (дата звернення: 29.10.2026).
29. ollama [Електронний ресурс] // Python Package Index. URL: https://pypi.org/project/ollama/ (дата звернення: 29.10.2026).
30. argparse. Parser for command-line options, arguments and subcommands [Електронний ресурс] // Python Documentation. URL: https://docs.python.org/3/library/argparse.html (дата звернення: 29.10.2026).
31. History and License [Електронний ресурс] // Python Documentation. URL: https://docs.python.org/3/license.html (дата звернення: 29.10.2026).